

F u n ç ã o

O que é?

É um Módulo e Software que realiza uma função específica. Ela é chamada pela assinatura (entenda-se neste momento por nome da função) dentro do algoritmo. Pode receber dados de entrada (argumentos) e retornar um “único” valor (adaptada da ISO 24765)

A Função permite **dividir** o algoritmo **em partes** e cada uma tem um nome específico.

A **diferença** entre função e procedimento é que a **função** pode retornar valor e o **procedimento** não. A linguagem Pascal possui as duas estruturas (*function* e *procedure*) em Linguagem C não.

A utilização da função permite que cada **ação** (Ex: ler, calcular e apresentar) do programa é escrita separada do código principal. Caso, não usarmos a estrutura de função todas as ações ficam no bloco principal do algoritmo. Exemplo:

Início

```
Inteiro IDADE, ANO
Escreva (“Informe o ano de seu nascimento”)
Leia (ANO)
IDADE = 2021-ANO
Escreva (“Sua idade é”, IDADE)
```

Fim

Cada função tem um nome, assim, quando precisar realizar aquela função, no exemplo, a leitura do ano, deve indicar no algoritmo e procurar o nome da função.

Para que serve?

- Reuso
- Manutenção
- Organização

O REUSO do código evita repetir dentro do algoritmo as linhas que executam a mesma função. Neste exemplo, no momento que for exibir a idade, chamaremos a função **apresentar()**.

Início

```
Inteiro IDADE, ANO
Escreva (“Informe o ano de seu nascimento”)
Leia (ANO)
IDADE = 2021-ANO
Escreva (“Sua idade é”, IDADE)
Escreva (“Agora vamos exibir de novo...”)
Escreva (“Sua idade é”, IDADE)
```

Fim

Pode ser substituído pela função apresentar()

Observamos, que podemos criar uma função para escrever a Idade do usuário, sem repetir o mesmo trecho dentro do código. Exemplo de reuso de código.

A Manutenção é mais fácil quando a alteração é feita em um único lugar, dentro da função, e repercute por todo o código, facilita, sem a necessidade de rever todas as linhas do algoritmo e fazer a alteração em várias linhas.

O uso da função auxilia na organização do código e facilita a manutenção.

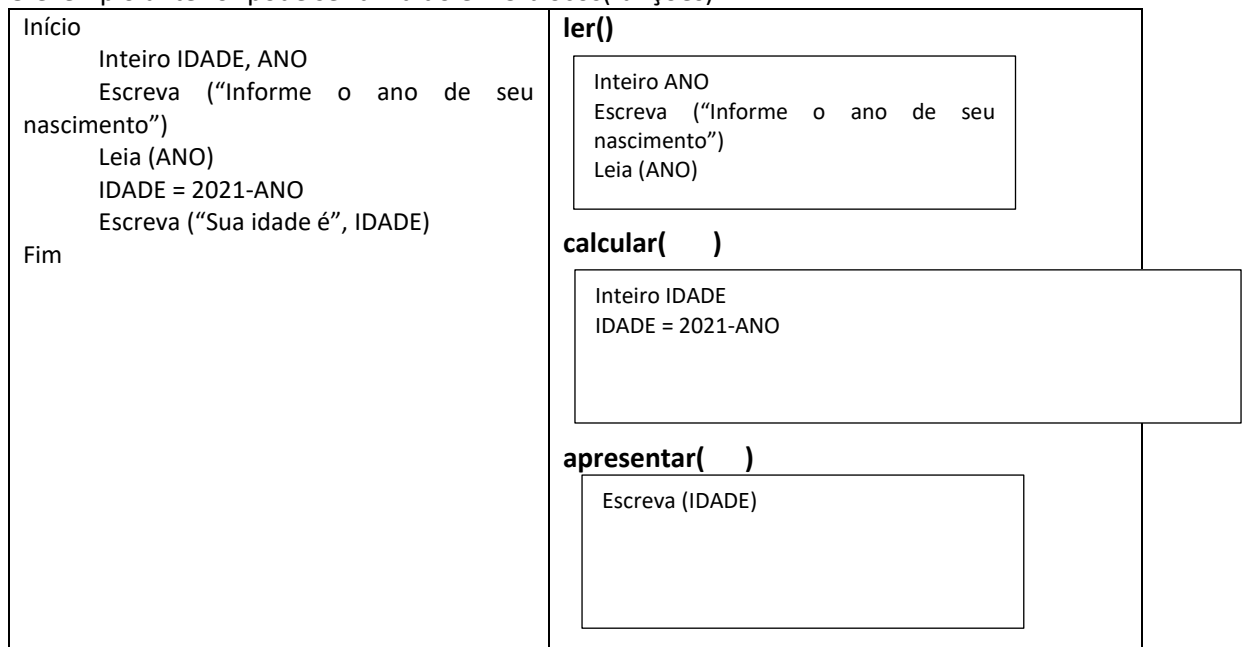
Como criar?

Vale algumas dicas:

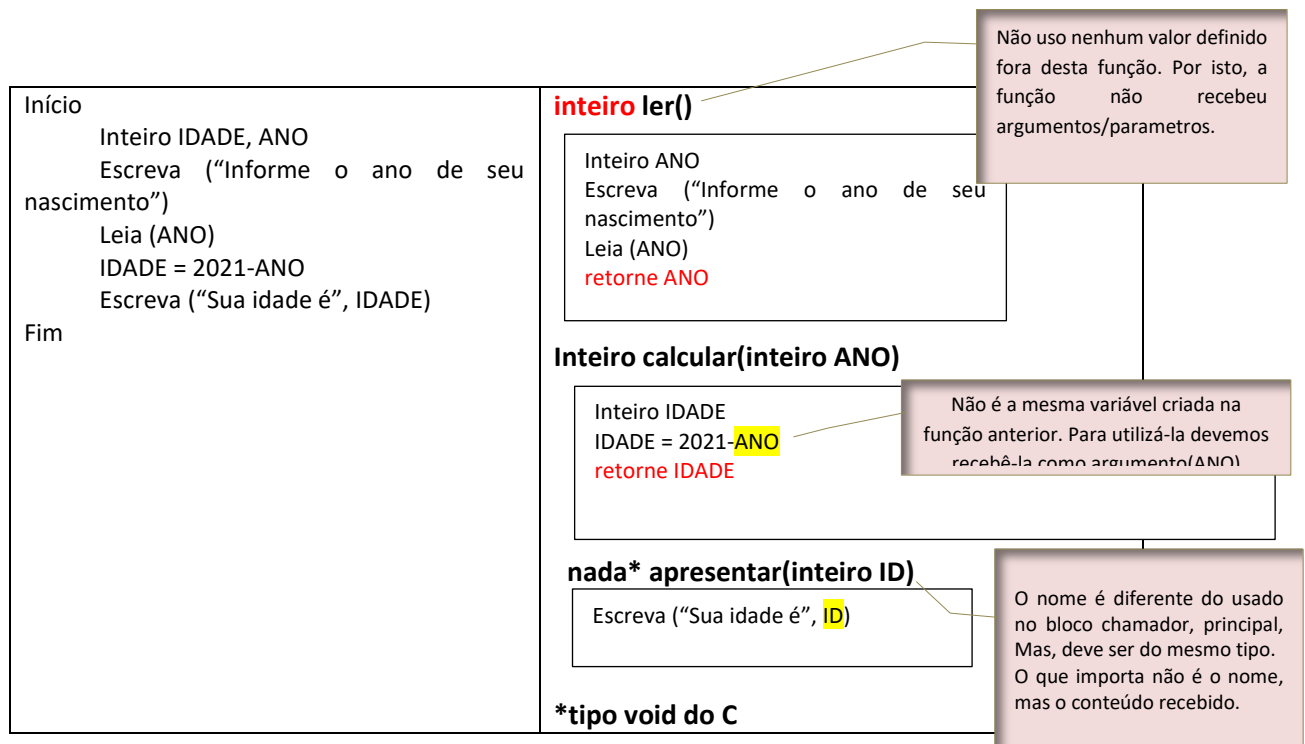
Dica 1: Lembrar qual objetivo da função (Modulo de software que realiza uma função específica).

Dica 2: Fazer um rascunho dos blocos de ação (funções) que devem ser criados.

O exemplo anterior pode ser dividido em 3 blocos(funções).



Dica 3: Lembrar que a variável que foi criada na função só pode ser utilizada dentro da função.



Não esquecer das perguntas

1ª Há algum valor que preciso usar fora da função? Se a resposta foi "sim", então, a função **retorna um valor**.

2ª Há algum valor a ser usado na função que foi definido fora da função? Se a resposta foi "sim", então, a função **recebe como argumento/parâmetro**. Lembre-se do mesmo tipo da variável criada no bloco chamador.

Dica 4: Dificuldades que podem ser encontradas na criação de uma função.

1. Por onde começar?

a. Procure ler o problema primeiro e pensar como o problema poderia ser quebrado em partes. Por exemplo, muitos problemas precisam de um programa que leia dados do usuário, faça cálculos e os apresente na tela. Então, é comum dividir o programa em uma parte de leitura, uma de cálculo e uma de apresentação. Se cada parte ainda ficar muito grande ou se essa parte gerar mais de um dado, então ela deve ser quebrada em mais partes.

b. Uma vez que você tenha dividido o problema (e o seu futuro algoritmo) em partes, pense como estas partes devem se relacionar.

2. Consegui pensar nas funções que meu algoritmo terá e sei como relacioná-las. O que faço agora?

a. Uma boa ideia é definir como cada função irá realizar sua tarefa. Por exemplo, se a função serve para ler um dado do usuário, defina as instruções **escreva** e **leia**.

3. As funções que criei precisam de parâmetros?

a. Uma vez que você saiba como a função deve realizar sua tarefa, você deve se perguntar: há algum dado a ser usado nesta função que foi definido fora dela? Se sim, você precisa de um parâmetro.

4. As funções que criei precisam retornar algum valor?

a. Para responder a esta pergunta, você deve se perguntar: há algum dado definido nesta função que será usado fora dela (em outra função)? Se sim, é preciso retornar um valor.

b. E se eu perceber que há mais de um dado definido nesta função e que serão usados em outro lugar (em outra função)? Então, você deve dividir esta função em duas (ou mais).

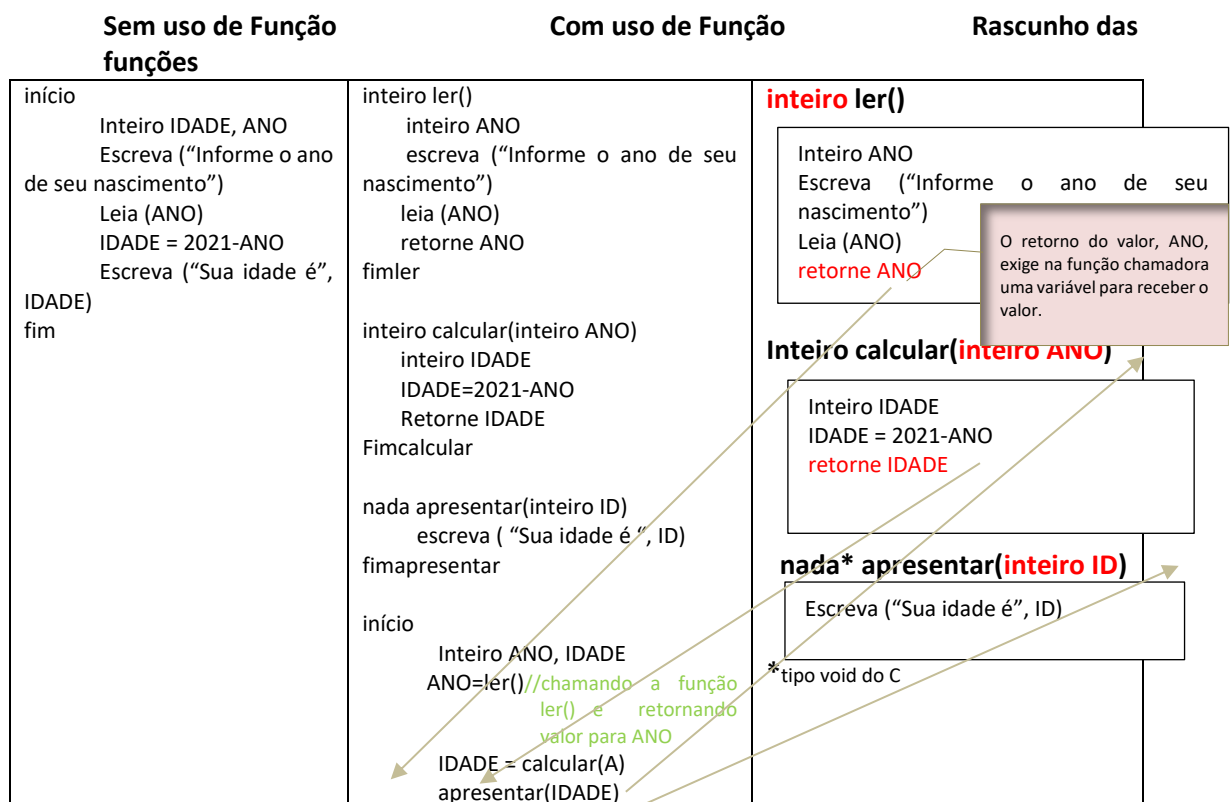
5. Criei cada função. Agora como faço para relacioná-las?

a. Tudo começa no bloco **início-fim**. A partir dele, você chamará o nome de cada função. Se a função chamada precisa de um dado externo, então você precisa passar esse valor como argumento.

b. Se a função a ser chamada retornar um valor, então, por enquanto, sempre atribua o valor a uma variável.

Como usar as funções?

Continuamos, com o exemplo para descobrir a idade da pessoa no ano atual.



	escreva ("Mostre a idade novamente") apresentar(IDADE) fim	
--	--	--

Recapitulando...

1º Lembre-se de retirar do enunciado do exercício as ações que possuem um objetivo específico. No exemplo, ler o ano, calcular a idade e apresentar a idade.

2º Crie um rascunho (retângulo no caderno) para cada função, dar um nome a função e definir o que cada uma faz.

3º Agora, é a hora de fazer a pergunta:

- Há algum valor da função que será usado fora dela? Se, sim, ela deve retornar um valor.

4º Definir o tipo da função está relacionado com o tipo da variável que será retornada. Se a função não retorna nenhum valor o tipo dela é "nada", em C, identifica o tipo *void*.

5º Outra pergunta importante: Dentro da função a algum valor que foi declarado fora da função? Se, sim, ela deve receber este valor, como argumento/parâmetros. O parâmetro armazenar o valor que vem de fora da função. Ex: inteiro apresentar (**ID**)

6º Agora, vamos criar o esboço da função principal, responsável em chamar as funções criadas. Como fazer para chamá-las no bloco principal?

5º Não esqueça sempre que chamo uma função, ela faz ou retorna algo(valor). A princípio vamos sempre atribuir o valor recebido da função a uma variável.